

AULA 12

O SLAM paga a conta da deriva

Etapa 6 · terça, 06/10/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

A aresta map -> odom é o erro da odometria, com o sinal trocado.

Não é uma transformada qualquer: é exatamente o número que vocês mediram como $\dot{\theta}$, publicado por quem conseguiu descobri-lo olhando para fora.

Cenas dos últimos episódios: onde paramos

- O sistema inteiro subindo com um comando, e o YAML lido e descartado em silêncio
- O mundo mínimo publicando /scan e /odom, sem física e sem janela
- A deriva da odometria medida: 0,57 m ao fim de 28 m, com pico de 0,95 m
- E uma frase que ficou em aberto: "map -> odom está, neste momento, mentindo"

Três peças voltam hoje, e a terceira é o que a aula resolve

- Aula 9 — ruído se cancela com repetição; viés não. O SLAM é a resposta ao viés
- Aula 11 — a árvore de TF, quatro arestas, cada uma com um dono
- Hoje UMA DELAS TROCA DE DONO
- Aula 11 — a deriva medida. Hoje ela vira a correção que o SLAM publica

O que você sai sabendo fazer hoje

- Provar que a sua árvore de transformadas está completa e sem aviso
- Ligar o SLAM ao seu robô acertando as quatro linhas que importam
- Explicar por que o placeholder tem de sair quando o SLAM entra
- Salvar o mapa construído e dizer, com um número, quanto ele acertou

A aula está ordenada por custo de perda

- 1. A sua árvore fecha? — a entrega é sexta — não pode cair
- 2. O SLAM assume a aresta — entrega em 15/10 — não pode cair
- 3. Salvar e medir o mapa — entrega em 15/10 — não pode cair
- 4. O que a navegação acrescenta — só no próximo trabalho — volta na Aula 13

O bloco 1 não é revisão: é pré-requisito do bloco 2.

O SLAM lê a sua TF para saber onde o laser estava quando mediu. Com a árvore quebrada ele sobe sem reclamar e não produz mapa nenhum.

PARTE 1

A sua árvore fecha?

Com o seu sistema no ar, agora

```
cd "$PB_WS" && source install/setup.bash
```

```
ros2 run tf2_tools view_frames
```

```
ros2 run tf2_ros tf2_echo map base_footprint
```

Uma árvore só, de map até os seus sensores. Sem órfãos e sem dois odom.

O que cada sintoma quer dizer

- PDF vazio -> o launch não está rodando. TF é fluxo, não arquivo
- Duas árvores -> falta uma junta; procure o frame sem pai
- Dois odom -> dois nós publicam a mesma aresta. Tire um
- base_footprint órfão -> falta odom -> base_footprint, quem integra a odometria

O SLAM não conserta TF errada.

Ele é construído em cima dela. Quem entra no bloco 2 com a árvore quebrada passa a aula olhando uma tela vazia e culpando o SLAM.

PARTE 2

O SLAM assume a aresta

Um comando, e o mapa começa a crescer

```
ros2 launch aula12_slam slam.launch.py  
ros2 launch aula12_slam slam.launch.py deriva:=6.0    # erro maior, correção maior
```

No RViz2 o mapa do SLAM cresce por cima do mundo real, que fica em cinza por baixo.

A diferença estrutural é UMA linha

aula12-slam/launch/slam.launch.py

```
IncludeLaunchDescription(  
    PythonLaunchDescriptionSource(  
        os.path.join(mundo_pkg, 'launch', 'mundo.launch.py')),  
    launch_arguments={'map_odom': 'false',      # <- o placeholder sai  
                      'deriva': deriva,  
                      'rviz': 'false'}.items(),  
),
```

O static_transform_publisher que publicava map -> odom como identidade tem de sair. Quem assume a aresta agora é o slam_toolbox.

Dois publicadores da mesma aresta não dão erro. Dão árvore quebrada.

Se você esquecer o `map_odom:=false`, o laser treme na tela e o `view_frames` mostra o estrago. É o defeito mais caro deste trabalho.

As quatro linhas que ligam o SLAM ao SEU robô

aula12-slam/config/slam.yaml

```
slam_toolbox:
  ros__parameters:
    odom_frame: odom          # de onde vem a odometria
    map_frame: map            # a aresta que ele vai PUBLICAR
    base_frame: base_footprint # a raiz do seu URDF
    scan_topic: /scan         # o sensor
```

Todo o resto do arquivo é o padrão do slam_toolbox e serve como está. Errar o base_frame é o modo mais comum de o SLAM subir calado e não entregar nada.

Ver a correção acontecer

```
ros2 run tf2_ros tf2_echo map odom      # a correção, crescendo  
ros2 topic echo /deriva                  # o erro que ela está corrigindo
```

Os dois números andam juntos. Esse é o fecho da semana passada.

A correção é a deriva, com o sinal trocado.

O robô não pode corrigir a odometria com mais odometria. O SLAM compara o /scan de agora com o mapa que vem construindo, e daí deduz quanto errou.

Fechamento de laço: o que a odometria nunca consegue fazer

- Quando o robô passa de novo por onde já esteve, o SLAM reconhece o lugar
- E a correção DÁ UM SALTO — não cresce suave, pula
- A odometria sozinha nunca faz isso: ela não tem como saber que já esteve ali
- Se o seu cenário é um corredor sem volta, o laço nunca fecha
- Isso é um achado sobre o seu projeto, não um defeito do SLAM

PARTE 3

Salvar e medir o mapa

Salvar, com o launch AINDA rodando

```
mkdir -p "$PB_DIR/maps" && cd "$PB_DIR/maps"  
ros2 run nav2_map_server map_saver_cli -f mapa
```

Gera o par mapa.pgm + mapa.yaml. O mapa vive na memória do nó: sem ninguém publicando, não há o que salvar.

Medir: o que normalmente não dá para fazer

```
cd /tmp/PBRoboticos_prof_dacio/exemplos/aula12-slam  
python3 comparar_mapas.py "$PB_DIR/maps/mapa.yaml"
```

```
cobertura    : 62.4% do mundo foi explorado  
concordancia: 96.1% das células conhecidas batem
```

Num robô real você não tem a verdade. Aqui o mundo é sintético, então ela existe.

Cobertura e concordância são coisas diferentes, de propósito

- Cobertura: quanto do mundo o robô chegou a explorar
- Concordância: quanto do que ele explorou bate com a verdade
- 30% de cobertura com 100% de concordância = mapa certo e incompleto
- 100% de cobertura com 70% de concordância = mapa completo e torto
- São problemas diferentes, com causas diferentes. Um número só esconderia os dois

A origem do mapa não é a origem do mundo.

O quadro map nasce onde o ROBÔ começou — aqui, em (2, 4). Errar esse desconto faz um mapa correto parecer péssimo, e é a primeira coisa a conferir quando o número vier baixo.



"Parece bom" não é resposta. De novo.

É a Aula 9 noutro objeto: declarar a régua antes de olhar o resultado, e desconfiar do agregado que esconde onde falhou.

PARTE 4

O que a navegação acrescenta

O piloto do mundo mínimo é o navegador mais burro que funciona

- Anda reto. Gira quando fecha na frente. Trinta linhas
- E explora o mundo inteiro sozinho
- Agora que existe mapa, a lista do que falta a ele fica concreta

Duas das quatro linhas passaram a ser possíveis hoje

- Não sabe onde está -> localização contra o mapa [o mapa é de hoje]
- Não sabe para onde vai -> um objetivo, como ação [as ações são da Aula 8]
- Não vê além do laser -> custos global e local [o mapa é de hoje]
- Não se recupera ao travar -> comportamentos de recuperação

O objetivo do Nav2 é uma ação.

Com feedback periódico e cancelamento — exatamente as da Aula 8. Navegação não é assunto novo: é a montagem de peças que já estão no lugar.

Tarefa da semana: construir, salvar e MEDIR o mapa do seu cenário

- O QUE FAZER: a correção map -> odom vindo do SLAM, não de um placeholder
- O QUE USAR: as quatro linhas do slam.yaml, o map_saver_cli, o comparar_mapas.py
- COMO ADEQUAR: é o SEU cenário que o SLAM mapeia — o comparador usa a sua versão como verdade
- E a métrica é sua decisão: qual concordância basta para o seu projeto? Declare antes

Como saber que terminou

```
ros2 launch aula12_slam slam.launch.py &      # deixe explorar uns 2 minutos
sleep 120

ros2 run tf2_tools view_frames                  # uma arvore, map no topo
ros2 run tf2_ros tf2_echo map odom --once     # a correcao, diferente de zero
ros2 run nav2_map_server map_saver_cli -f mapa
```

Guarde as saídas junto do número do comparador.

Desafio: quanto de mapa o fechamento de laço vale?

- Rode duas vezes, 3 minutos cada: deriva:=1.0 e deriva:=8.0
- Salve os dois mapas e compare a concordância
- A concordância caiu proporcionalmente à deriva, ou o SLAM absorveu o erro?
- Em que ponto ele deixa de dar conta?
- E quanto vale calibrar a odometria de um robô que vai rodar SLAM de qualquer jeito?

Para a próxima aula, 13/10 — percepção

- Segmentação por instância e o módulo veicular
- O mapa diz onde o robô está; a percepção diz o que há em volta
- Chegue com o mapa salvo e medido
- Ele é a base das duas entregas seguintes